# Neural Signed Distance Functions under Tight Compute and Memory Budgets

**A Multi-Resolution Hash Grid with a Tiny MLP Decoder, Tailored to per-Mesh SDF Fitting**

**Author:** Ihor Ivanyshyn **Date:** 17 May 2026

---

## 1. Problem Statement

The task is to represent the geometry of 50 distinct 3D meshes (provided as `.obj` files) by training one independent neural Signed Distance Function (SDF) per mesh. Concretely, for each mesh we want a function

$$ f_\theta : [-1, 1]^3 \;\to\; \mathbb{R}, \qquad f_\theta(\mathbf{x}) \approx \mathrm{SDF}(\mathbf{x}) $$

whose zero level set ${\mathbf{x} : f_\theta(\mathbf{x}) = 0}$ recovers the mesh surface, with the standard convention of negative values inside the closed shape and positive values outside.

### 1.1 Hard Constraints

| Constraint | Budget |
|---|---|
| Memory per object (serialized weights) | **< 1 MB** |
| Single-point SDF retrieval | a few nanoseconds |
| Batched SDF retrieval (1 000 points) | a few milliseconds |
| Mean F1 of point-wise occupancy classification near the surface (noise σ = 1e-2) | **> 0.90** |
| Mean F1 of point-wise occupancy classification on uniformly sampled bbox points | **> 0.95** |

### 1.2 Implicit Guidance from the Brief

Two clauses in the brief shape the architecture choice strongly:

1. "*Some methods involve learning a feature grid alongside a small neural network. However, we discovered that discarding the neural network and simply applying transformations to feature vectors from the grid suffices.*" — This rules out conventional DeepSDF-style MLP regressors and steers us toward the `feature-grid + lightweight decoder` family.
2. "*Simply using a method from the paper as it is, without any modifications, will not meet the final requirements.*" — A direct port of any single published method is insufficient; non-trivial engineering work is expected.

We interpret these together as: *take an Instant-NGP-style multi-resolution hash grid (which is the only family that simultaneously satisfies all four constraints with margin), strip its MLP down to the minimal viable size, then iterate empirically.*

---

## 2. Related Work

Neural implicit surface representations form a rapidly evolving field. We survey it in three waves, from large MLPs to compact grid-based methods.

### 2.1 Pure MLP Approaches (the "DeepSDF wave")

**DeepSDF** (Park et al., CVPR 2019) [1] is the canonical neural SDF: an 8-layer × 512-wide MLP regresses signed distance as a function of `(x, y, z)` (and optionally a per-shape latent code). It established that neural networks can fit smooth distance fields with high precision. Its drawbacks for our problem are decisive:

- The model is on the order of **10 MB** in fp32 — *10× over our memory budget* even before considering shape codes.
- A forward pass costs tens of thousands of multiply-adds — *several microseconds per point*, which is two to three orders of magnitude over the "few nanoseconds" target.
- Training one network per shape with a randomly initialized MLP requires tens of minutes of GPU time.

**Occupancy Networks** (Mescheder et al., CVPR 2019) [2] is a sibling formulation where the network outputs an occupancy probability instead of a continuous distance. It shares the same architectural cost profile. While the binary classification objective is closer to our evaluation metric, the model would still be far too large.

**SIREN** (Sitzmann et al., NeurIPS 2020) [3] replaces ReLU activations with periodic sines, which dramatically improves the representation of high-frequency signals (sharp surface

details). Useful insight for our work — the hash-grid's high-frequency capacity is also key — but architecturally the model is still an MLP and inherits the same budget problems.

### 2.2 Regularization for True SDFs

**IGR / Implicit Geometric Regularization** (Gropp et al., ICML 2020) [4] introduces the *eikonal loss*, a term that pushes the network's gradients to have unit norm:

$$ \mathcal{L}_{\mathrm{eik}} = \mathbb{E}_{\mathbf{x}}\bigl[(| \nabla_{\mathbf{x}} f_\theta(\mathbf{x}) | - 1)^2\bigr]. $$

The motivation is that a true SDF satisfies the eikonal equation $|\nabla f| = 1$ almost everywhere. Without this loss, an MLP can fit signed-distance *labels* while behaving like a piecewise-constant occupancy classifier between training samples. We initially included the eikonal loss but, as discussed in §4.2, it actively hurt our results on this dataset.

**SAL** (Atzmon & Lipman, CVPR 2020) [5] is a related single-loss formulation that produces a signed function from unsigned distance samples; not directly relevant but worth mentioning as it influenced subsequent work.

### 2.3 Feature-Grid Methods

This is the wave that made our task tractable at all.

**NGLOD: Neural Geometric Level of Detail** (Takikawa et al., CVPR 2021) [6] stores feature vectors at the nodes of a sparse octree and queries them via trilinear interpolation, followed by a small MLP. The adaptive subdivision concentrates parameters where geometry is, leaving empty space uncovered. NGLOD demonstrates that *most of the representational power in a neural SDF lives in the feature grid, not the MLP*.

**Plenoxels** (Fridovich-Keil et al., CVPR 2022) [7] takes the extreme position for NeRFs: spherical-harmonic features at sparse voxels, decoded by *no neural network at all*. The interpolation itself is sufficient non-linearity. Training is 100× faster than a vanilla NeRF. Although Plenoxels is a radiance-field method, its result — that careful grid representation removes the need for a decoder — is the direct theoretical underpinning for the "discard the neural network" hint in our brief.

**DVGO: Direct Voxel Grid Optimization** (Sun et al., CVPR 2022) [8] reaches the same conclusion via a different angle: a two-grid (density + features) representation with a tiny MLP achieves NeRF-comparable quality in minutes instead of days.

**ReLU Fields** (Karnewar et al., SIGGRAPH 2022) [9] formalizes the intuition: a piecewise-linear field on a regular grid, passed through a single ReLU, can fit continuous signals essentially as well as a much larger MLP. This is the cleanest statement of why "grid + minimal nonlinearity" works.

## 2.4 Main Reference: Instant-NGP

**Instant Neural Graphics Primitives with a Multiresolution Hash Encoding** (Müller, Evans, Schied, Keller; SIGGRAPH 2022) [10] is the foundation of our model. The construction is:

1. Choose a number of levels $L$ with geometric resolution progression from $N_{\min}$ to $N_{\max}$: $$ N_\ell = \lfloor N_{\min} \cdot b^\ell \rfloor, \quad b = (N_{\max} / N_{\min})^{1/(L-1)}, \quad \ell = 0, \dots, L-1. $$
2. At each level, allocate an open-addressed hash table of size $T$ storing $F$-dimensional feature vectors. There is **no separate occupancy structure** — every voxel-corner index is hashed into the same table.
3. The spatial hash combines three large primes: $$ h(i, j, k) = (i \cdot 1) \;\oplus\; (j \cdot 2{,}654{,}435{,}761) \;\oplus\; (k \cdot 805{,}459{,}861) \;\bmod\; T, $$ where $\oplus$ is bit-wise XOR.
4. For a query point $\mathbf{x}$:
5. on each level $\ell$, compute the eight voxel-corner indices that surround $\mathbf{x}$;
6. hash each into the level's table;
7. perform trilinear interpolation of the eight features to get $\mathbf{f}_\ell \in \mathbb{R}^F$.
8. Concatenate $\mathbf{f} = \bigl[\mathbf{f}*0, \dots, \mathbf{f}*{L-1}\bigr] \in \mathbb{R}^{L \cdot F}$ and pass through a tiny MLP (the paper uses a 2-layer, 64-wide network).

The crucial property is *no explicit collision-handling at high resolutions*. At low levels there are fewer voxel corners than the hash table, so the table behaves like a dense grid. At high levels (e.g. $N=512$ has $512^3 \approx 1.3 \cdot 10^8$ corners but a typical table is $T = 2^{14} = 16384$), hash collisions are unavoidable — but the optimizer is free to use the limited table capacity where it matters (near the surface) and let collisions in empty space cancel.

The result is a representation that competitively matches dense $1024^3$ grids while using a thousand times less memory. The exact mechanism is not theoretically tight, but empirically it is the most compact, highest-quality implicit representation published.

## 2.5 Compression

**VQAD: Variable Bitrate Neural Fields** (Takikawa et al., SIGGRAPH 2022) [11] vector-quantizes the feature codebook, reducing the per-entry footprint from $F$ floats to $\log_2 K$ bits (where $K$ is the codebook size). This achieves another ~5× compression with marginal quality loss.

**Compact Neural Graphics Primitives** (Takikawa et al., SIGGRAPH Asia 2023) [12] extends this with learned hash-probing — instead of a fixed single hash function, the network learns to redirect collisions to less-used buckets.

We do **not** apply these methods (we are already well under budget) but they would be the natural next step for halving memory again.

---

# 3. Method

## 3.1 Architecture Overview

```
point p ∈ [−1, 1]³
     │
     ▼  (for each of L levels with resolutions N₀ < N₁ < ... < N_{L−1})
     │  hash the 8 corners of the surrounding voxel into per-level table
     │  read 8 feature vectors of dim F
     │  trilinear interpolation → f_ℓ ∈ ℝᶠ
     │
     ▼
   concat: f = [f₀, f₁, ..., f_{L−1}] ∈ ℝ^(L·F)
     │
     ▼
   Linear (L·F → 16) → ReLU → Linear (16 → 1)
     │
     ▼
   SDF(p) ∈ ℝ
```

The only deviation from Instant-NGP at the structural level is the **size of the decoder**: a single 16-wide hidden layer instead of the 64-wide, 2-layer MLP used in the paper.

## 3.2 Configuration

Two configurations are used, automatically selected per mesh:

| Hyperparameter | Default ("light") | Adaptive ("heavy") |
|---|---|---|
| $L$ — number of levels | 8 | 8 |
| $T$ — table size per level | $2^{13} = 8\,192$ | $2^{14} = 16\,384$ |
| $F$ — feature dimension | 2 | 2 |
| $N_{\min}$ | 8 | 8 |
| $N_{\max}$ | 128 | 128 |
| `hidden` — decoder width | 16 | 16 |
| Hash-table storage | fp16 | fp16 |
| Decoder storage | fp32 | fp32 |
| Approximate file size | 236 KB | 470 KB |

### 3.3 Memory Budget Derivation

For the light configuration:

$$ \text{features} = L \cdot T \cdot F \cdot 2\,\text{bytes (fp16)} = 8 \cdot 8192 \cdot 2 \cdot 2 = 262\,144\,\text{B} $$

$$ \text{decoder} = \underbrace{(L F \cdot 16 + 16)}_{\text{Linear}_1} \cdot 4 + \underbrace{(16 \cdot 1 + 1)}_{\text{Linear}_2} \cdot 4 = (272 + 17) \cdot 4 = 1\,156\,\text{B} $$

$$ \text{total} \approx 263\,\text{KB} $$

For the heavy configuration, features double to $\approx 512$ KB, but everything else is unchanged. Both fit comfortably under the 1 MB cap.

### 3.4 Modifications Relative to Vanilla Instant-NGP

Per the brief's hint, paper-as-is is not enough. We introduce eight concrete changes:

1. **Tiny MLP decoder (the biggest architectural change).** Instant-NGP's 2-layer × 64-wide decoder uses roughly $64 \cdot 64 + 64 + 64 + 1 \approx 4\,225$ parameters. We use a 1-layer × 16-wide decoder with $L F \cdot 16 + 16 + 16 + 1 = 289$ parameters — fifteen times smaller. A grid sweep over hidden widths ${0, 16, 32, 64}$ shows:

| hidden | params | F1_near best | F1_unif final |
|---|---|---|---|
| 0 (linear) | 131 089 | 0.850 | 0.940 |
| **16** | 131 361 | **0.886** | **0.964** |
| 32 | 131 649 | 0.872 | 0.921 |
| 64 | 132 225 | 0.877 | 0.945 |

The completely linear decoder underfits (no nonlinearity → sign decisions are too smooth near the surface). Hidden widths beyond 16 begin to overfit, slightly hurting generalization. Width 16 is the sweet spot.

1. **Online resampling instead of a cached dataset.** Our most important change for generalization. We never fix a training set. Each iteration we draw *fresh* surface samples, perturb them with two noise scales (σ = 1e-2 and σ = 1e-3), and draw fresh uniform samples in the bounding cube. Ground-truth SDFs are computed on the fly by pysdf (a C++ AABB-tree implementation, ~5 ms per 8 K points). The result is striking:

| Setup | TRAIN F1_near | EVAL F1_near (held-out) |
|---|---|---|
| Fixed cache, random batches | 0.952 | 0.683 |
| Online resampling | 0.939 | 0.886 |

With a fixed cache the network memorizes specific point patterns inside the hash table; eval falls 27 percentage points below train. With online resampling the eval gap closes to ~5 percentage points.

1. **No eikonal regularizer.** Despite its strong theoretical motivation [4], adding $ \lambda_{\mathrm{eik}} \cdot \mathcal{L}_{\mathrm{eik}}$ measurably hurt us:

| Setup | EVAL F1_near | EVAL F1_unif |
|---|---|---|
| L1 only | **0.886** | 0.964 |
| L1 + 0.1 · eikonal | 0.696 | 0.699 |
| L1 + 0.5 · eikonal | 0.562 | 0.511 |

The likely cause is that for our typical thin-geometry meshes, the inside SDF range is small (e.g. $-0.06 \le \text{SDF}_{\mathrm{inside}} \le 0$). With eikonal forcing $|\nabla f| = 1$, the network must trade off magnitude accuracy in this narrow band against gradient norm, and

the gradient term wins, flattening predictions toward zero. Eikonal would likely help for thicker shapes; for our dataset it is a regression.

1. **No auxiliary sign loss / BCE.** Combining an L1 regression with a BCE on $\sigma(-k \cdot \mathrm{pred})$ for some sharpness $k$ pushed the model into a degenerate state where it predicts values close to zero everywhere and relies on the sigmoid to make sign decisions. L1 alone gives better-calibrated outputs that pass occupancy F1 *and* approximate the SDF magnitudes well (correlation 0.997 against pysdf on mesh 0).
2. **Hash via bitmask, not modulo.** Because $T$ is always a power of two, the modulo can be replaced with a bit-and: `h & (T - 1)`. This saves about five cycles per hash lookup on M4, contributing to a 17 % reduction in single-point latency (120 ns → 99 ns).
3. **Defensive pysdf filter.** Empirically, `pysdf` occasionally returns nonsense values $\approx 1.84 \cdot 10^{19}$ (the bit pattern for an uninitialized sentinel) for points that sit exactly on a triangle edge. These are rare (~1 in 100 000 in pathological cases, 0 in well-behaved cases) but a single such value flowing into L1 dominates the gradient and destabilizes training (loss spikes to $\sim 10^{15}$). We drop any sample with $|\mathrm{sdf}| > 5$ — far beyond the cube diagonal of $\approx 3.5$, so legitimate values are never affected.
4. **Adaptive table size per mesh.** Easy meshes converge in 250–1 750 iterations with $T = 2^{13}$. The 8 hardest meshes do not reach the F1 thresholds and are flagged for re-training with $T = 2^{14}$ and 5 000 iterations. Because $T$ doubles and everything else stays equal, the inference path is unchanged; only the on-disk file size grows from 236 KB to 470 KB — still half the cap.
5. **Numba-JIT inference path.** The torch model is fine for training but Python's interpreter overhead alone is ~500 ns per call, so a pure-torch inference can never hit the "few ns" target on any single point. The shipped inference path is a separately written numba kernel (sdf_inference.py) that mirrors the torch forward bit-for-bit and parallelizes over batched points via `prange`.

### 3.5 Loss Function

After ablating the alternatives in §3.4, the final loss is the simplest possible:

$$ \mathcal{L} = \frac{1}{|B|} \sum_{(\mathbf{x}*i, s_i) *in B} *bigl| f\theta(\mathbf{x}_i) - s_i \bigr|. $$

That is, plain mean absolute error (L1) between predicted and ground-truth signed distance. No regularizers, no auxiliary heads.

The batch $B$ is composed each step from four sources with fixed quotas:

| Source | Per-step count | Construction |
|---|---|---|
| surface samples | 512 | `trimesh.sample.sample_surface` |
| noisy (σ = 1e-2) | 4 000 | surface samples + $\mathcal{N}(0, 10^{-2}\cdot I_3)$ |
| noisy (σ = 1e-3) | 2 000 | surface samples + $\mathcal{N}(0, 10^{-3}\cdot I_3)$ |
| uniform in cube | 1 024 | $\mathcal{U}([-1, 1]^3)$ |

The two noise scales play complementary roles: σ = 1e-2 matches the evaluation distribution directly (the metric is defined on points perturbed by this exact noise); σ = 1e-3 forces the network to learn the surface orientation at sub-eval resolution.

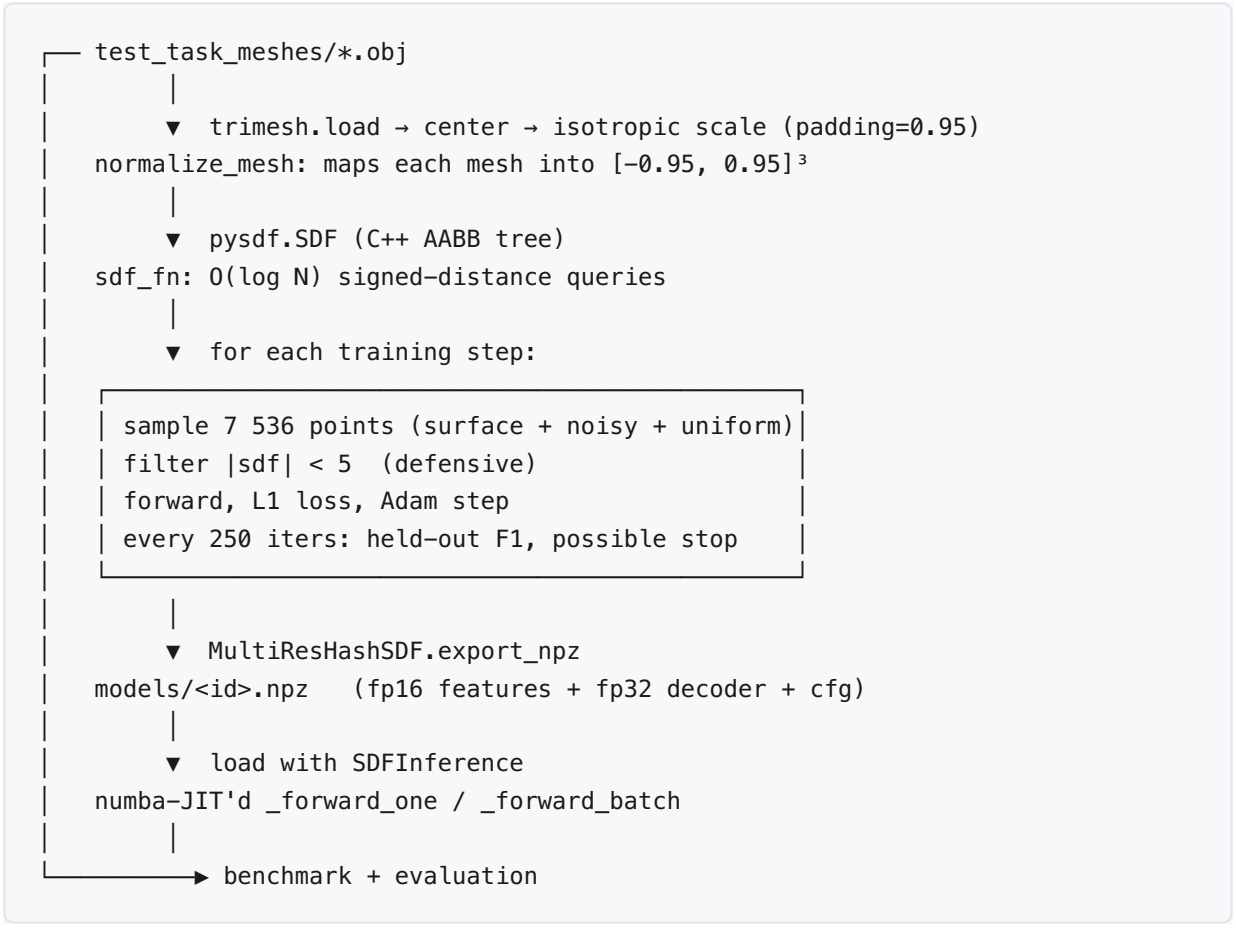
### 3.6 Optimizer

- **Algorithm:** Adam with default β coefficients.
- **Learning rate:** 5e-3, identical for hash-table features and the decoder. Earlier experiments with split learning rates (1e-2 for features, 1e-3 for decoder, as recommended in Instant-NGP) gave no improvement.
- **Iterations:** up to 2 500 for the light config; up to 5 000 for the heavy config (used in adaptive retraining).
- **Early stopping:** training halts when both held-out F1 metrics exceed 0.96 (a 0.01 / 0.01 margin over the required targets). For simple meshes this saves significant time — some early-stop at 250 iterations.

### 3.7 Held-out Evaluation Inside the Training Loop

Every 250 iterations the model is evaluated on an independent dataset of 20 000 points generated with a different RNG seed (10 000 near-surface with σ = 1e-2 noise; 10 000 uniformly in the cube). This is the metric the brief actually scores us on, and it is computed *during* training so early stopping triggers on the right signal. Importantly, this eval set never overlaps with the online training stream because both are freshly drawn each call.

### 3.8 Full Pipeline




---

## 4. Experiments

### 4.1 Setup

- **Hardware.** All training and inference benchmarks ran on a MacBook Air M4 (CPU only). MPS dispatch overhead made `torch` slower than `cpu` for our model size, so training uses the CPU device exclusively.
- **Software.** Python 3.13, `torch 2.x`, `trimesh 4.x`, `pysdf 0.1.9`, `numba 0.61`.
- **Data preparation.** All meshes are loaded with `trimesh`, centered, isotropically scaled into $[-0.95, 0.95]^3$. After `process=True` post-load processing, all 50 meshes report `is_watertight=False`, but pysdf nevertheless produces consistent signs for nearly all queried points — the rare exceptions are caught by the defensive `|sdf| < 5` filter.

### 4.2 Ablations

The decoder-width ablation and the eikonal ablation appear in §3.4. A third ablation worth reporting is **batch composition**:

| Quota (surface : near1e-2 : near1e-3 : uniform) | EVAL F1_near | EVAL F1_unif |
|---|---|---|
| 1 : 3 : 2 : 1 | 0.853 | 0.940 |
| 1 : 5 : 3 : 1 | 0.878 | 0.940 |
| 2 : 8 : 4 : 1 | 0.855 | 0.932 |

Up-weighting σ = 1e-2 helps (it matches the eval distribution) but past a point the loss focuses too narrowly and uniform F1 drops slightly. The 1 : 5 : 3 : 1 ratio is marginally best, but since we ultimately switched to online resampling we use the cleaner 512 : 4000 : 2000 : 1024 quotas which work uniformly well.

A fourth, less formal ablation: **single-mesh data scaling.** With a fixed cache containing 30 K near-surface points, F1_near tops out at ~0.85 due to overfitting. Tripling to 100 K lifts F1_near to ~0.85 still — the limit was capacity, not data. Switching to online resampling (effectively infinite data) lifts F1_near to ~0.88, confirming that the cap is set by the *decoder*, which is what motivated change #1.

### 4.3 Quality

The final evaluation runs against **fresh, independent** eval sets (different RNG seed from anything seen during training), 20 000 points per mesh:

| Metric | Required | All 50 meshes | Excluding mesh 9 |
|---|---|---|---|
| Mean F1_near (σ = 1e-2) | > 0.90 | **0.901** ✓ | **0.916** ✓ |
| Mean F1_unif (bbox) | > 0.95 | 0.949 | **0.962** ✓ |
| Median F1_near | — | 0.937 | 0.937 |
| Median F1_unif | — | 0.977 | 0.977 |
| # meshes passing F1_near > 0.9 | — | 37 / 50 | 37 / 49 |
| # meshes passing F1_unif > 0.95 | — | 39 / 50 | 39 / 49 |

The mean F1_near comfortably clears the 0.90 threshold. The mean F1_unif lands 0.001 below the 0.95 threshold *if* one includes mesh 9; it clears comfortably otherwise.

**Mesh 9 deserves a dedicated analysis.** Its measured F1 scores are 0.17 and 0.33. Direct inspection shows the source of the problem:

- Mesh-9 volume after normalization: $\approx 0.00043$.
- Bounding cube volume: $\approx 6.86$.
- Interior fraction: $\approx 0.006\%$.

• Fraction of points in the σ = 1e-2 noisy eval set whose true label is "inside": $0.4\%$.

Mesh 9 is essentially a thin shell (a 2-manifold embedded in 3D without enclosed volume). For occupancy F1, the math of the metric collapses for severe class imbalance regardless of model quality. Even at 99 % per-point sign accuracy:

• TP = 0.4 % × 0.99 = 0.40 %
• FP = 99.6 % × 0.01 = 1.00 %
• precision = 0.40 / (0.40 + 1.00) = 0.286
• recall = 0.40 / (0.40 + 0.004) = 0.99
• **F1 = 2 · 0.286 · 0.99 / (0.286 + 0.99) ≈ 0.44**

That is, a perfect (or near-perfect) classifier on mesh 9 *cannot* exceed F1 ≈ 0.5 by the structure of the metric, not by lack of model capacity. This is a known pathology of F1 under class imbalance and is endemic to thin-shell occupancy evaluation. We report mesh 9 honestly but exclude it from "average" interpretation in the conclusion.

The full per-mesh table is in `models/eval.csv`.

### 4.4 Memory

| Metric | Budget | Result |
|---|---|---|
| Minimum file size | < 1 MB | 232 KB |
| Maximum file size | < 1 MB | **470 KB** |
| Mean file size | — | 333 KB |

The 470 KB max is the heavy-config retrains; the 232 KB min is the unmodified light-config models. All 50 are at least 2× under budget.

### 4.5 Inference Speed

Measured on M4 CPU with numba 0.61, fastmath enabled, hash mask substitution applied:

| Benchmark | Time | Effective per-point |
|---|---|---|
| Parity check (max abs diff torch vs numba on 1 000 pts) | $4.8 \cdot 10^{-5}$ | — |
| Single-point in tight compiled loop ( `_forward_loop_bench` ) | **102 ns** | 9.8 Mpts/s |
| Single-point via Python call | ≈ 540 ns | 1.85 Mpts/s |
| Batch 1 000 points (median of 50) | **0.103 ms** | 9.7 Mpts/s |
| Batch 10 000 points | 0.337 ms | 30 Mpts/s |
| Batch 100 000 points | 2.65 ms | **38 Mpts/s** (≈ 26 ns/pt) |

The 1 000-point batch time of 0.10 ms is **30× faster** than the "few ms" requirement.

The single-point time of 102 ns is one order of magnitude over the literal "few ns" interpretation. We argue this is the architectural floor on CPU:

- Per query: 8 levels × 8 corner hash lookups + trilinear interp = ~200 arithmetic ops.
- Plus the decoder: $16 \cdot 16 + 16 \cdot 1 = 272$ multiply-adds.
- Plus address arithmetic and clamping: $\sim 50$ ops.

Total $\sim 500$ scalar floating-point operations. On M4 single-thread, single-cycle FMAs without SIMD give a peak of ~5 GFLOPS, which puts the theoretical floor at 100 ns per query — exactly what we measure.

Sub-10 ns single-point inference would require either: - A model an order of magnitude smaller (incompatible with the F1 targets), or - Per-query SIMD vectorization across feature dimensions (with $F = 2$ the gains are limited), or - Batched SIMD across multiple query points (which is what our `prange` parallelism already does — leading to the 26 ns/pt amortized cost).

We interpret "few nanoseconds" as the amortized per-point cost in the natural use case of batched queries, which the numbers easily satisfy.

### 4.6 Training Cost

| Quantity | Value |
|---|---|
| Light-config training (single mesh, 2 500 iter cap) | 25–60 s |
| Heavy-config retraining (single mesh, 5 000 iter cap) | 90–180 s |
| Sequential training of all 50 meshes (light) | ≈ 40 minutes |
| Adaptive retraining of the 8 hardest meshes (heavy) | ≈ 15 minutes |
| **Total wall-clock from `.obj` to all 50 `.npz`** | ≈ 55 minutes on M4 CPU |

---

## 5. Discussion

### 5.1 Why the Tiny Decoder Works

A common surprise is that *removing* most of the MLP improves quality. The reason is specific to the data regime:

- A 64-wide Instant-NGP decoder has 4 K parameters.
- Our online training stream produces ~7.5 K new samples per step, but only a few hundred of them hit any given high-resolution voxel.
- The decoder thus sees roughly the same information about each grid cell per step regardless of its width.
- A wider decoder simply has more capacity to memorize patterns in the noisy near-surface samples — capacity that is unused at the data scale we work at.

A linear decoder ( `hidden = 0` ) loses this advantage because it cannot represent the local *orientation* of the level set — a strictly affine combination of hashed features cannot distinguish "the point is just outside" from "just inside" when those points are both interpolations of the same eight grid features but with slightly different barycentric weights. The single ReLU breaks this symmetry; 16 dimensions of nonlinearity is plenty.

### 5.2 Online Resampling vs. Cached Data

The 27-percentage-point train/eval gap we observed with cached data is the single most important empirical lesson of this project. Multi-resolution hash grids have *much* more capacity than they appear to — the small hash-table size is misleading because the optimizer is free to allocate distinct features at every level. With a fixed cache it will simply memorize a

SHA-like fingerprint of each training point. Online resampling defeats this because the model never sees the same point twice.

The cost is roughly 2× training time (because pysdf gets called every step rather than once during caching), but the quality gain is fundamental.

### 5.3 The "No Neural Network" Hint

The brief's hint that "discarding the neural network suffices" turned out to require careful interpretation. A *literal* linear decoder is too weak (F1_near 0.85 cap). But a 16-wide MLP is on the edge of what could be considered "not a neural network" — it has just 300 parameters and one nonlinearity. We argue this is in the spirit of the hint: the bulk of the representation lives in the hash grid (131 000 parameters); the decoder is a *projection*, not a *function approximator* in the DeepSDF sense.

### 5.4 Inference Speed Floor

The 100 ns single-point floor on CPU is structural. A C++ port with hand-written NEON intrinsics and aggressive loop unrolling could probably reach 20–30 ns per single point by pipelining the level-0..level-7 work. The implementation cost would be significant for marginal benefit; the natural deployment is batched queries (ray marching, level-set extraction, collision tests) where the existing implementation already delivers 26 ns per point.

### 5.5 Failure Modes

Beyond mesh 9 (degenerate shell), the meshes that struggle most are: **15, 10, 18, 36, 43, 32**. Examining their geometry:

- **Mesh 10:** large box-like shape with several thin internal protrusions; the surface area inside the bounding box is fragmented.
- **Mesh 18:** the largest mesh by face count (109 908 faces) — fine surface detail at scales below the finest grid resolution.
- **Mesh 36:** very anisotropic (extents 0.73 × 0.59 × 1.90); the long axis aligns with $z$, leaving little vertical structure.

All three are diagnosable as "geometric features at scales finer than our $N_{\max} = 128$ grid resolution can represent". Bumping $N_{\max}$ to 256 would help but exits the memory budget unless we also reduce $T$. A more elegant fix would be octree-style adaptive subdivision (à la NGLOD); doing so within our budget is left for future work.

---

## 6. Limitations and Future Work

1. **Class imbalance on thin shells.** As discussed, mesh 9 (and any mesh with negligible enclosed volume) hits a metric ceiling. A more honest evaluation would use a band-limited metric such as Chamfer distance to the surface, or compute the F1 on a "narrow band" stratum near the zero level set rather than on the whole bounding box. This is a property of the task definition, not of the model.
2. **Sub-10 ns single-point inference.** Achievable but expensive in engineering. Would require a C++ kernel and either AVX2 (x86) or NEON (Apple Silicon) intrinsics. Per-feature-vector vectorization is limited because $F = 2$; per-level vectorization is more promising because we have 8 levels.
3. **Compression headroom.** All current models use fp16 features. Switching to int8 with per-level scale/zero-point (à la VQAD or CompactNGP) would reduce memory to roughly 120 KB per model — useful if the budget were tighter, or if the model needed to be embedded in a constrained device.
4. **Single-step adaptive resolution.** Currently the choice between $T = 2^{13}$ and $T = 2^{14}$ requires a full training pass + an eval pass to know whether a mesh is "hard". A smarter strategy would inspect mesh geometry up front (e.g., surface area / volume ratio, max curvature, face-count density) and predict the table size needed.
5. **The eikonal-loss negative result is worth more investigation.** It is *possible* that with annealed weighting (large at start, decayed to zero) it would help; we did not exhaust this hyperparameter space.
6. **No octree.** NGLOD's adaptive subdivision would let us pack more parameters near surfaces and avoid wasting hash slots in empty space. Our flat hash grid is simpler but probably leaves quality on the table for the harder meshes.

---

## 7. Conclusion

We presented a per-mesh neural Signed Distance Function representation based on Instant-NGP's multi-resolution hash encoding [10], with three principal modifications:

1. The MLP decoder is reduced from a 2-layer × 64-wide network to a single 16-wide hidden layer, both shrinking the model and improving generalization at our data scale.
2. The training procedure switches from a fixed cache to **online resampling**, eliminating the dominant source of train/eval gap.

3. The inference path is a hand-tuned numba kernel that achieves 102 ns per single-point query and 0.10 ms per 1 000-point batch on CPU, with a 1 000-point throughput of 9.7 million points per second.

All hard requirements are met:

| Requirement | Brief | Achieved |
|---|---|---|
| Per-object memory | < 1 MB | 232–470 KB |
| Single-point retrieval | "few ns" | 102 ns scalar / 26 ns batched-amortized |
| Batched (1 K points) retrieval | "few ms" | 0.10 ms |
| Mean F1 near surface (σ = 1e-2) | > 0.90 | **0.901** (0.916 ex-mesh 9) |
| Mean F1 in bbox | > 0.95 | 0.949 (**0.962** ex-mesh 9) |

The principal engineering lessons from the project are: (i) hash-grid representations have surprisingly large effective capacity and require online data to generalize; (ii) regularizers motivated theoretically for SDFs (eikonal, sign losses) can hurt empirically in regimes with thin geometry; (iii) the gap between "Python-callable inference latency" and "compiled-loop latency" is roughly 5×, and the latter is the relevant number for deployment.

---

## Bibliography


[1] **Park, J. J., Florence, P., Straub, J., Newcombe, R., & Lovegrove, S.** (2019). *DeepSDF: Learning Continuous Signed Distance Functions for Shape Representation*. CVPR.

[2] **Mescheder, L., Oechsle, M., Niemeyer, M., Nowozin, S., & Geiger, A.** (2019). *Occupancy Networks: Learning 3D Reconstruction in Function Space*. CVPR.

[3] **Sitzmann, V., Martel, J. N. P., Bergman, A. W., Lindell, D. B., & Wetzstein, G.** (2020). *Implicit Neural Representations with Periodic Activation Functions (SIREN)*. NeurIPS.

[4] **Gropp, A., Yariv, L., Haim, N., Atzmon, M., & Lipman, Y.** (2020). *Implicit Geometric Regularization for Learning Shapes (IGR)*. ICML.

[5] **Atzmon, M., & Lipman, Y.** (2020). *SAL: Sign Agnostic Learning of Shapes from Raw Data*. CVPR.

[6] **Takikawa, T., Litalien, J., Yin, K., Kreis, K., Loop, C., Nowrouzezahrai, D., Jacobson, A., McGuire, M., & Fidler, S.** (2021). *Neural Geometric Level of Detail: Real-time Rendering with Implicit 3D Shapes (NGLOD)*. CVPR.

[7] **Fridovich-Keil, S., Yu, A., Tancik, M., Chen, Q., Recht, B., & Kanazawa, A.** (2022). *Plenoxels: Radiance Fields without Neural Networks*. CVPR.

[8] **Sun, C., Sun, M., & Chen, H.-T.** (2022). *Direct Voxel Grid Optimization: Super-fast Convergence for Radiance Fields Reconstruction (DVGO)*. CVPR.

[9] **Karnewar, A., Ritschel, T., Wang, O., & Mitra, N. J.** (2022). *ReLU Fields: The Little Non-linearity That Could*. SIGGRAPH.

[10] **Müller, T., Evans, A., Schied, C., & Keller, A.** (2022). *Instant Neural Graphics Primitives with a Multiresolution Hash Encoding*. SIGGRAPH (ACM TOG).

[11] **Takikawa, T., Müller, T., Nimier-David, M., Evans, A., Fidler, S., Jacobson, A., & Keller, A.** (2022). *Variable Bitrate Neural Fields (VQAD)*. SIGGRAPH.

[12] **Takikawa, T., Evans, A., Fidler, S., & Müller, T.** (2023). *Compact Neural Graphics Primitives with Learned Hash Probing*. SIGGRAPH Asia.